FUNDAÇÃO UNIVERSIDADE DE CAXIAS DO SUL
ÁREA DO CONHECIMENTO DE CIÊNCIAS EXATAS E ENGENHARIAS
FBI4013AA - Programação de Computadores II
PROVA I, 14 DE SETEMBRO DE 2022

Nome [redacted]

1) (3.3 pontos) Fazer uma função escrita na linguagem C com três parâmetros do tipo string. Essa função deve separar as letras e os dígitos da string informada como primeiro parâmetro, colocando as letras (maiúsculas ou minúsculas) em uma string (segundo parâmetro) e os dígitos em outra (terceiro parâmetro). Caracteres que não sejam dígitos ou letras não devem ser colocados em nenhum dos dois vetores.

```
Exemplos de chamada: // conteudo das variáveis após as chamadas ==>   slet        sdig
  separa("ABC   123",slet,sdig);                          //     "ABC"        "123"
  separa("A1B2D8F4",slet,sdig);                           //     "ABDF"       "1284"
  separa("Rua  X,  984  ap  72",slet,sdig);               //     "RuaXap"     "98472"
  separa("  ",slet,sdig);              // ""        ""
```

VAi SER NA DOR :) testartodoalfabeto :)

2) (3.3 pontos) Fazer uma função escrita na linguagem de programação C que receba uma lista de 100 palavras em uma matriz char, possivelmente contendo repetições, e verifique qual a palavra que ocorre mais vezes, retornando em um segundo parâmetro a palavra que mais ocorre, e como valor de retorno da função o número de ocorrências da mesma. Considere que as palavras têm no máximo 20 caracteres.

Protótipo: int rep(char lista[100][21], char palrep[21])

DOBLE DOUBLE

3) (3.4 pontos) Fazer um programa escrito na linguagem de programação C que leia números (float) armazenando os mesmos em uma matriz com QTLIN linhas por QTCOL colunas e escreva para cada linha a diferença entre o maior e o menor número da linha. Considere que a matriz pode conter valores positivos e negativos. Criar e usar uma função com dois parâmetros: uma matriz e um número de linha, e retorne a diferença entre o maior e o menor número da linha. Considerar que QTLIN e QTCOL estão definidos pela cláusula #define. Abaixo um exemplo para uma matriz 4 linhas por 6 colunas.

```
|3.3  2.5  6.7  2.2  3.5  5.4|
|2.3  1.1  4.1  9.3  9.9  1.1|
|9.0  4.8  4.4  3.7  7.5  6.8|
|8.2  3.7  6.4  5.1  2.7  6.3|
```

Deveria escrever:

```
Linha 0, diferença: 4.5       // 6.7 – 2.2 = 4.5
Linha 1, diferença: 8.8       // 9.9 -1.1 = 8.8
Linha 2, diferença: 5.3       // 9.0 – 3.7 = 5.3
Linha 3, diferença: 5.5       // 8.2 – 2.7 = 5.5
```

---- BOA PROVA ----

____/____/______) [redacted]

```
#INCLUDE <Stdio.h>
#iNCLUDE <STRiNG.h>
~~INT~~ void LOUD(CHAR P[]){
    iNT i,D=0, L=0;
    CHAR SDiG[100], SLEt[100];
    fOR(i=0; P[i]!='\0'; i++){
        if(P[i]=="1"||P[i]=="2"||P[i]=="3"||P[i]=="4"||P[i]=="5"||
           P[i]=="6"||P[i]=="7"||P[i]=="8"||P[i]=="9"||P[i]=="0"){
            SDiG[D]=P[i];
            D++;
        }ELSE{ if(P[i]=="a"||P[i]=="b"||P[i]=="c"||P[i]=="d"||
                  P[i]=="e"||P[i]=="f"||P[i]=="g"||P[i]=="h"||
                  P[i]=="i"||P[i]=="j"||P[i]=="k"||P[i]=="l"||
          P[i] == "m"||P[i]=="n"||P[i] == "o"||P[i] == "p"||P[i] == "q"||
          P[i] == "r"||P[i]=="s"||P[i] == "t"||P[i] == "u"||P[i] == "v"||
          P[i] == "w"||P[i]=="x"||P[i] == "y"||P[i] == "z"||P[i] == "A"||
          P[i] == "B"||P[i]=="C"||P[i] == "D"||P[i] == "E"||P[i] == "F"||
          P[i] == "G"||P[i]=="H"||P[i] == "I"||P[i] == "J"||P[i] == "K"||
          P[i] == "L"||P[i]=="M"||P[i] == "N"||P[i] == "O"||P[i] == "P"||
          P[i] == "Q"||P[i]=="R"||P[i] == "S"||P[i] == "T"||P[i] == "U"||
          P[i] == "V"||P[i]="W"||P[i] == "X"||P[i] == "Y"||P[i] == "Z"){
          SLEt[L]=P[i];
          L++;
        }
    }
    RETURN ~~0~~;
}
```

"fazer uma função com **três** parâmetros.

'1'

(aspa simples)

2.5

Colocar '\0' no final das strings.

2 5.9

___/___/___

```
# INCLUDE <StDIO.h>
# INCLUDE <STRING.h>
INT RPT(CHAR LIS[][]){
   INT S, J, L, C, RQ=0 , CC, CCC
   CHAR RP[21]
   FOR(L=0; L<100; L++){
      FOR (J=L+1 ; J<99; J++){   S=0
         FOR(C=0; C<20; C++){
            IF(LIS[L][C]!='\0' && LIS[J][C]!='\0' &&
               LIS[L][C]==LIS[J][C]){
                  S++;
            }
         IF(S>RQ){
            RQ=S;
            FOR(CC=0; LIS[L][CC]!='\0'; CC++){
               RP[CC]=LIS[L][CC];
               IF(RP[CC+1]!='\0'){
                  FOR(CCC=CC+1; RP[CCC]!='\0'; CCC++){
                     RP[CCC='\0';
                  }
               }
            }
         }
      }
   }
   RETURN RP;
   RETURN RQ;
}
```

100

21

'retornando em um segundo parâmetro

conta quantas letras são iguais?

Isso encontra a maior palavra, e não a que mais ocerre!

2

[redacted]

___/___/___

```
#iNCLUDE <StDiO.h>
#iNCLUDE <StRiNG.h>
#DEFiNE QtLiN 10
#DEFiNE QtCOL 10
iNt ALM(iNt M[][], iNt L){
     U
    DOBLE MiN, MAX, RSt;
    MiN = M[L][0];
    MAX = M[L][0];
    iNt i;
    FOR(i=0; i<QtCOL; i++){
        iF (M[L][i]>MAX){
            MAX = M[L][i];
        }
        iF(M[L][i]<MiN){
            MiN = M[L][i];
        }
    }
    RSt = MAX - MiN;
    REtURN RSt;
}
VOiD MAiN(){
    iNt L, C, i;    DOUBLE M[QtLiN][QtCOL];
    FOR (L=0; L<QtLiN; L++){
        FOR(C=0; C<QtCOL; C++){
            SCANF("%F", M[L][C]);
        }
    }
    FOR(i=0; i<QtLiN; i++){
        PRiNtF("LiNHA %d, DiFERENSA:%F", i, ALM(M[][], i));
    }
}
```

3.4